Aula 06 · Exercício: preveja antes de executar

Disciplina: Paradigmas de Linguagens de Programação
Tema: Tipos de Dados (Sebesta, Cap. 6)
Aluno: Gabriel Santos Furlanette
R.A.: 24000873-2
Curso: Engenharia de Software · Unicesumar

Para cada trecho: **(a)** qual é a saída? **(b)** o problema é detectado na **compilação**, na **execução** ou **nunca**?

Trecho 1 · JavaScript

```
console.log(0.1 * 3 === 0.3);
console.log(9007199254740993);
```

- **(a) Saída:** `false` / `9007199254740992`
- **(b) Nunca.** Em JS todo número é `double` (IEEE 754). `0.1 * 3` vale `0.30000000000000004`, por isso a comparação dá `false`. Acima de 2^{53} o double não representa todos os inteiros, e `...993` é arredondado para `...992`. Nenhum erro ou aviso é gerado.

Trecho 2 · Python

```
p = "maçã"
print(len(p), len(p.encode()))
```

- **(a) Saída:** `4 6`
- **(b) Nunca.** `len(p)` conta caracteres (4). `p.encode()` gera bytes UTF-8, onde `ç` e `ã` ocupam 2 bytes cada: 1 + 2 + 1 + 2 = 6. Não é erro, é a diferença entre texto e bytes.

Trecho 3 · Go

```
var b byte = 255
b++
fmt.Println(b)
```

- **(a) Saída:** `0`
- **(b) Nunca.** `byte` é um inteiro sem sinal de 8 bits (0 a 255). `255 + 1` estoura e volta para `0` silenciosamente. O compilador só reclamaria de uma constante fora da faixa, como `var b byte = 256`.

Trecho 4 · Java

```
int[] v = new int[3];
System.out.println(v[0]);
System.out.println(v[3]);
```

- **(a) Saída:** `0` / `ArrayIndexOutOfBoundsException`
- **(b) Execução.** Arrays de `int` são inicializados com zero, então `v[0]` imprime `0`. O índice 3 não existe (vai de 0 a 2) e a JVM verifica os limites em tempo de execução, lançando a exceção.

Trecho 5 · Rust

```
let s = String::from("oi");
let t = s;
println!("{} {}", s, t);
```

- **(a) Saída:** não compila — `borrow of moved value: s`
- **(b) Compilação.** `let t = s;` **move** a posse da `String` para `t`, e `s` deixa de ser válida. O borrow checker barra o uso de `s` antes de o programa existir.

Trecho 6 · C

```
union { int i; float f; } u;
u.f = 1.0f;
printf("%d\n", u.i);
```

- **(a) Saída:** `1065353216`
- **(b) Nunca.** A `union` faz `i` e `f` dividirem a mesma memória. Os bits do float `1.0f` (`0x3F800000`) lidos como `int` valem `1065353216`. O C não verifica qual campo foi escrito por último.

Resumo

#	Linguagem	Saída	Detectado
1	JavaScript	false / 9007199254740992	Nunca
2	Python	4 6	Nunca
3	Go	0	Nunca
4	Java	0 / exceção	Execução
5	Rust	não compila	Compilação
6	C	1065353216	Nunca

Conclusão

Quanto mais forte e estática a verificação de tipos da linguagem, mais cedo o problema aparece. Rust detecta na compilação, Java detecta na execução, e JavaScript, Python, Go e C deixam o programa rodar com um resultado inesperado, sem nenhum aviso. Todos os trechos foram executados para conferir as previsões.